

Sistema Web de Reservas de Espacios y Recursos

Documentación técnica — Módulo 2, Arquitectura Cloud

Describe un sistema monolítico modular desarrollado en Python puro (sin frameworks web) con base de datos MySQL, desplegado en cPanel (g21.origenet.cl).

1. Funcionalidades del Sistema por Rol

Usuarios Regulares

- **Autenticación:** Registro único con nombre, correo y contraseña; inicio de sesión posterior con sesión firmada.
- **Visualización de Disponibilidad:** Listado completo de todos los espacios o recursos configurados en la plataforma.
- **Gestión de Reservas:** Selección del espacio y rango de fecha/hora con validación de disponibilidad en tiempo real: el sistema rechaza reservas que se solapen con otra reserva activa del mismo espacio.
- **Cancelación Autónoma:** Anulación con un solo clic desde la lista de reservas del usuario, restringida estrictamente a reservas propias (validado en la capa de negocio y en la consulta SQL).

Administradores

- **Gestión de Inventario:** Creación de nuevos espacios o recursos para ponerlos a disposición del público. El formulario solo se muestra al rol admin y la restricción se valida además en el servidor.
- **Seguridad:** Mismo esquema que todos los usuarios: contraseñas con hash y salt, y sesiones firmadas criptográficamente.

Nota de alcance: la gestión de usuarios y el monitoreo global de todas las reservas NO están implementados como vistas de la aplicación; esa supervisión se realiza por phpMyAdmin sobre la base de datos. Desarrollar un panel administrativo integrado queda registrado como propuesta de mejora (sección 9).

2. Arquitectura del Sistema y Reglas de Diseño

El sistema implementa un monolito modular con Patrón de Arquitectura en Capas. La consigna original propone microservicios; las restricciones del entorno (Python puro sin frameworks, hosting compartido cPanel sin Docker ni procesos independientes) llevaron a adoptar un único

despliegue físico cuyos límites internos están trazados como si fueran microservicios. El objetivo que persiguen los microservicios —mantenibilidad, escalabilidad y módulos que evolucionan sin romper al resto— se cumple mediante alta cohesión, bajo acoplamiento y modularidad, y la separación estricta de responsabilidades facilita la refactorización futura hacia microservicios reales.

Capas Definidas

- **Capa de Presentación:** Maneja el enrutamiento y la interacción con el usuario mediante `views.py`, `router.py` y `wsgi.py`, más la gestión de sesiones/cookies y las plantillas HTML.
- **Capa de Lógica de Negocio (Services):** Contiene y aplica las reglas del dominio (usuarios, espacios, reservas). Es la interfaz pública de cada módulo y evita el acoplamiento directo entre ellos.
- **Capa de Acceso a Datos (Repositories):** Encapsula por completo las consultas SQL, asegurando que la lógica de negocio sea agnóstica a la base de datos subyacente.

Principio de Encapsulamiento de Servicios

Se estableció una frontera de diseño crítica para mitigar el acoplamiento modular:

- **Prohibido:** Un módulo jamás puede consultar directamente el Repository o las tablas de la base de datos de un dominio ajeno.
- **Permitido:** Un módulo solo invoca el `service.py` (interfaz pública) de otro módulo.

Ejemplo real en el código: al crear una reserva, `reservas/service.py` valida que el espacio exista llamando a `espacios.service.obtener_espacio(espacio_id)`; nunca consulta la tabla `espacios` directamente. Este es el único punto de cruce entre módulos de dominio. Si mañana un módulo se extrajera como microservicio, bastaría reemplazar esa llamada de función por una llamada HTTP.

Flujo de una petición (ejemplo: crear una reserva)

1. El navegador envía POST `/reservas`; `app/wsgi.py` recibe la petición WSGI y la delega en el router.
2. El router despacha al handler registrado en `views.py` (presentación), que lee el formulario y verifica la cookie de sesión firmada.
3. El handler llama a `reservas.service.crear_reserva()`, que valida fechas, consulta a `espacios.service` y verifica solapamientos vía repository.
4. El repository ejecuta SQL parametrizado contra MySQL a través de `db.py`; la vista renderiza la plantilla HTML de respuesta.

3. Servicios Principales de la Lógica de Negocio

Tres servicios especializados operan bajo un principio estricto de aislamiento de dominio:

- **Servicio de Usuarios (usuarios/service.py):** Orquesta el ciclo de vida del usuario y la seguridad de acceso. Funciones clave: registrar() (creación, validación de integridad y hash de credenciales) y autenticar() (verificación de credenciales con comparación timing-safe y datos para la sesión firmada).
- **Servicio de Espacios (espacios/service.py):** Administra el inventario de recursos. Funciones clave: listar_espacios() (visibilidad pública), obtener_espacio() (consulta puntual, usada también por el módulo de reservas) y crear_espacio() (persistencia de nuevos recursos, operación restringida al rol de administrador).
- **Servicio de Reservas (reservas/service.py):** Gestiona la disponibilidad y el histórico de transacciones. Funciones clave: crear_reserva() (valida coherencia de fechas, existencia del espacio y disponibilidad temporal para evitar conflictos), listar_de_usuario() y cancelar_reserva() (anulación garantizando que solo el dueño puede cancelar, filtrando por usuario_id).

4. Matriz de Tecnologías Utilizadas

Tecnología	Uso y Ámbito de Aplicación
Python 3.11 (librería estándar)	Backend y lógica del sistema, sin frameworks web. Router, plantillas (string.Template), hashing y servidor de desarrollo resueltos con la stdlib.
WSGI / CGI	WSGI es la interfaz estándar entre servidor web y aplicación Python; en el hosting la app se ejecuta vía CGI (wsgiref.handlers.CGIHandler) sin modificar la lógica.
MySQL (InnoDB) / PyMySQL	Base de datos relacional con claves foráneas e índice de solapamiento; conector Python puro con consultas parametrizadas, vendorizado en vendor/pymysql/.
hashlib (pbkdf2_hmac) + hmac	Hash de contraseñas y firma/verificación de cookies de sesión, solo con librería estándar.
HTML / CSS (string.Template)	Interfaz gráfica de usuario accesible desde navegadores (PC, celular, tablet), renderizada en el servidor.
pytest + pytest-cov	Pruebas unitarias automatizadas con aislamiento de datos y reporte de cobertura (dependencia solo de desarrollo).
Locust	Pruebas de estrés y carga concurrente (dependencia solo de desarrollo).
cPanel + Apache + AutoSSL	Entorno de despliegue y administración de infraestructura; HTTPS activo mediante AutoSSL.

5. Entorno de Desarrollo Local

Prerrequisitos: Python 3.10 o superior (el código usa dataclasses y uniones de tipos modernas), servidor MySQL 5.7+ / MariaDB 10.3+ y Git.

Configuración inicial del entorno a través de la terminal de comandos:

```
# 1. Crear y activar entorno virtual
python -m venv .venv
source .venv/bin/activate # En Windows: .venv\Scripts\activate

# 2. Instalar dependencias del sistema
pip install -r requirements.txt -r requirements-dev.txt

# 3. Configurar variables de entorno
cp .env.example .env # Editar .env con credenciales locales de MySQL
y una SECRET_KEY propia

# 4. Crear e inicializar la base de datos local
mysql -u <usuario> -p -e "CREATE DATABASE reservas CHARACTER SET
utf8mb4;"
mysql -u <usuario> -p reservas < db/schema.sql
mysql -u <usuario> -p reservas < db/seed.sql

# 5. Levantar servidor de desarrollo
python -m app.devserver # http://localhost:8000
```

Credenciales de Prueba Definidas (datos del seed)

- **Administrador:** admin@demo.cl / Clave: Admin1234
- **Usuario Regular:** usuario@demo.cl / Clave: Usuario1234

Verificación

5. Abrir <http://localhost:8000> — dirige a /espacios.
6. Iniciar sesión con un usuario del seed y crear una reserva.
7. Intentar crear otra reserva solapada (debe rechazarse) y cancelar la primera.
8. Ejecutar `pytest tests/unit -v` — todas las pruebas deben pasar.

6. Control de Calidad y Pruebas Automatizadas

Pruebas Unitarias (pytest)

Estrategia orientada al aislamiento completo de la base de datos para validar la lógica pura de negocio: 34 pruebas en total, ejecución en menos de un segundo, sin conexiones reales a BD.

- **Comando de ejecución:** `pytest tests/unit -v` (con cobertura: `pytest --cov=app tests/unit`)

- **Aislamiento:** FakeRepository en memoria (listas simuladas) e inyección de dependencias; unittest.mock.patch para aislar el cruce entre módulos (reservas → espacios.service).
- **Patrones:** Assertions explícitas (un test, una razón de falla) y pytest.raises para validar las excepciones de dominio esperadas (EspacioOcupadoError, ReservaNoAutorizadaError, etc.).
- **Cobertura:** 100% en la capa de lógica de negocio (usuarios/service.py, espacios/service.py y reservas/service.py), que es el objetivo de las pruebas unitarias. La cobertura global del proyecto es 27%, porque la capa de presentación y la infraestructura (views, router, wsgi, repositories) se validan con las pruebas de carga y la verificación funcional, no con unitarias.

Pruebas de Estrés y Carga (Locust)

Garantizan la resistencia de la plataforma ante peticiones concurrentes simulando usuarios simultáneos contra la aplicación en ejecución.

- **Comando de ejecución:** locust -f tests/carga/locustfile.py --host http://localhost:8000 --headless -u 50 -r 5 -t 1m --csv resultados_carga (los parámetros -u/-r/-t ajustan usuarios, rampa y duración del escenario).
- **Flujo evaluado:** Usuario autenticado (inicia sesión vía POST /login y luego alterna tareas con pesos 3:2:1 — listar espacios, ver reservas propias y crear reserva).
- **Métricas medidas:** Tiempo de respuesta (mediana y percentiles), solicitudes por segundo (RPS / throughput) y tasa de fallos (fail rate).

7. Pipeline de Despliegue en cPanel (vía CGI)

Debido a las restricciones del hosting (sin soporte SSH ni Setup Python App / Passenger), la app WSGI se ejecuta vía CGI y las dependencias se empaquetan localmente (vendoring). El cambio no tocó la lógica de negocio.

Estructura del Artefacto

- **index.cgi:** Punto de entrada que carga el .env e inicializa wsgiref.handlers.CGIHandler para arrancar la aplicación WSGI.
- **.htaccess:** Fuerza la directiva ExecCGI, enruta todas las peticiones a index.cgi preservando PATH_INFO y restringe el acceso a archivos sensibles (.env, .py, .sql).
- **vendor/pymysql/:** Copia física del conector de base de datos (el hosting no dispone de pip), agregada dinámicamente al sys.path por el script de inicio.
- **info.cgi:** Script de sondeo temporal (versión de Python e intérpretes disponibles). Debe eliminarse del servidor al terminar el despliegue.

Checklist de Lanzamiento Crítico

9. **Base de Datos:** Crear la base de datos y el usuario desde cPanel asignando todos los privilegios. Importar schema.sql y seed.sql usando phpMyAdmin.

10. **Carga de Archivos:** Subir el comprimido del proyecto directamente a la raíz de public_html/ (document root del dominio), sin subcarpetas; subirlo a una subcarpeta produce errores 404.
11. **Variables de Entorno:** Habilitar la visualización de archivos ocultos en el File Manager, crear manualmente el archivo .env e ingresar las credenciales de producción (nunca se versiona en el repositorio).
12. **Permisos Críticos (CHMOD 755):** Asignar manualmente permisos 755 a index.cgi. Cada sobrescritura del archivo restaura los permisos a 644 de forma automática, provocando un Error 500 si no se reaplican.
13. **Ajuste del Intérprete (Shebang):** Modificar la primera línea de index.cgi para apuntar al Python de CloudLinux del hosting: #!/opt/alt/python311/bin/python3 (el Python 3.6 del sistema es insuficiente para el código).
14. **HTTPS:** En SSL/TLS Status ejecutar Run AutoSSL y verificar que el sitio responda por https://. Eliminar info.cgi del servidor.

8. Matriz de Seguridad y Operaciones

Vector de Riesgo Detectado	Estrategia de Mitigación Implementada
Fuga de Contraseñas	Hashing robusto (no cifrado reversible) mediante pbkdf2_hmac con SHA-256, 200.000 iteraciones y salt aleatorio único por usuario; verificación timing-safe con hmac.compare_digest.
Secuestro / Manipulación de Sesión	Diseño completamente stateless: cookies de sesión firmadas criptográficamente con HMAC-SHA256 acoplado a una SECRET_KEY; una cookie alterada se descarta.
Inyección SQL	Blindaje total en la capa de persistencia parametrizando de forma estricta todos los inputs con marcadores %s; nunca concatenación de strings.
Acceso no autorizado a operaciones	Autorización por rol (crear espacios solo admin) y por propiedad (cancelar reservas solo el dueño), validadas en servidor.
Exposición de Infraestructura	Eliminación obligatoria del archivo de diagnóstico info.cgi en producción; .htaccess bloquea el acceso directo a .env, .py y .sql.
Intercepción del Tráfico	HTTPS obligatorio en producción mediante AutoSSL de cPanel (con HSTS activo); secretos solo en variables de entorno.